Lab Report: MicroPython 2

Sensors and Input Components

Students: Yosi Brim & Elad Asbag
Supervisor: Dr. Yosef Golobtsov

March 22, 2026

Introduction

This laboratory explores the integration of various sensors and input components using MicroPython on an ESP32 microcontroller. Throughout the tasks, we implemented Pulse Width Modulation (PWM) for precise output control, utilized Analog-to-Digital Converters (ADC) to read external analog sensors, generated mathematical signal waveforms, and processed digital timing signals to build an interactive distance-measuring system.

---

1 Task 1: RGB LED Control via User Input

Description: In this task, we configured three PWM channels to control the Red, Green, and Blue pins of an RGB LED. The system prompts the user to input values between 0 and 255 for each color channel, which are then mapped to the corresponding PWM duty cycles.

Expected Result: The RGB LED should emit the exact mixed color corresponding to the three numeric values provided by the user in the serial terminal.

Circuit & Execution Proof:

Source Code:

```python
from machine import Pin, PWM
import sys

red_pin = PWM(Pin(12), freq=5000)
green_pin = PWM(Pin(13), freq=5000)
blue_pin = PWM(Pin(14), freq=5000)

def set_color(r, g, b):
    red_pin.duty(r * 4)
    green_pin.duty(g * 4)
    blue_pin.duty(b * 4)

print("--- RGB LED Control System ---")

while True:
    try:
        val_r = int(input("Enter RED value (0-255): "))
```

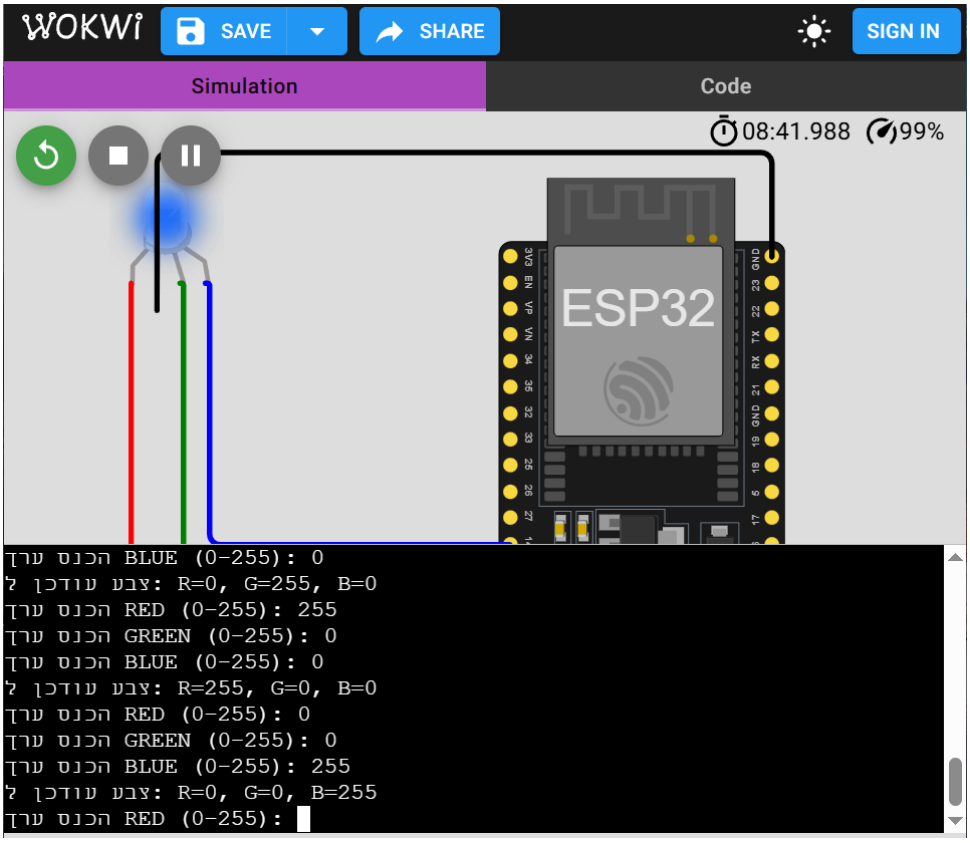

Figure 1: Wokwi simulation showing the RGB LED reacting to user input.

```
        val_g = int(input("Enter GREEN value (0-255): "))
        val_b = int(input("Enter BLUE value (0-255): "))

        if 0 <= val_r <= 255 and 0 <= val_g <= 255 and 0 <= val_b
            <= 255:
            set_color(val_r, val_g, val_b)
            print(f"Color updated: R={val_r}, G={val_g}, B={val_b
                }")
        else:
            print("Error: Values must be between 0 and 255!")

    except ValueError:
        print("Error: Please enter integers only.")
```

2 Task 2: Automatic Lighting Control (LDR)

Description: This task involves reading ambient light levels using a Light Dependent Resistor (LDR) connected to the ESP32's ADC. The software calculates a proportional value and adjusts a LED's brightness using PWM.

Expected Result: In total darkness, the LED should shine at maximum brightness. As the ambient light level increases, the LED's brightness should smoothly decrease until it completely turns off.

Circuit & Execution Proof:

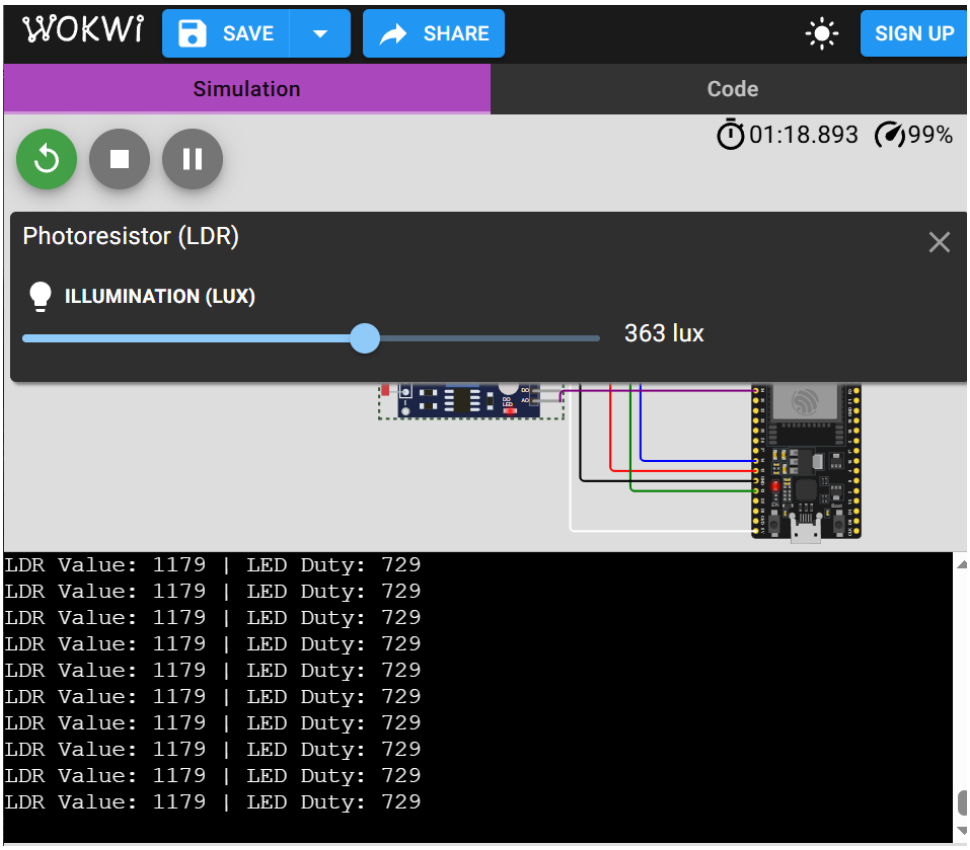

Figure 2: Simulation demonstrating the inverse relationship between LDR light level and LED duty cycle.

Source Code:

```
from machine import Pin, PWM, ADC
import time

led = PWM(Pin(15), freq=5000)
ldr = ADC(Pin(34))
ldr.atten(ADC.ATTN_11DB)

print("--- Auto Lighting System ---")

while True:
    light_value = ldr.read()
```

```python

    duty_cycle = int(light_value / 4)

    if duty_cycle < 0: duty_cycle = 0
    if duty_cycle > 1023: duty_cycle = 1023

    led.duty(duty_cycle)
    print(f"LDR Value: {light_value} | LED Duty: {duty_cycle}")
    time.sleep(0.1)
```

3 Task 3: Triangular Wave Generator (Breathing LED)

Description: The objective was to generate a triangular wave using PWM to create a smooth "breathing" effect on an LED. The program receives the desired frequency and amplitude from the user and calculates the necessary time delays.

Expected Result: The LED should smoothly increase in brightness up to the specified amplitude and then smoothly decrease back to zero (triangular wave, not sawtooth).

Circuit & Execution Proof:

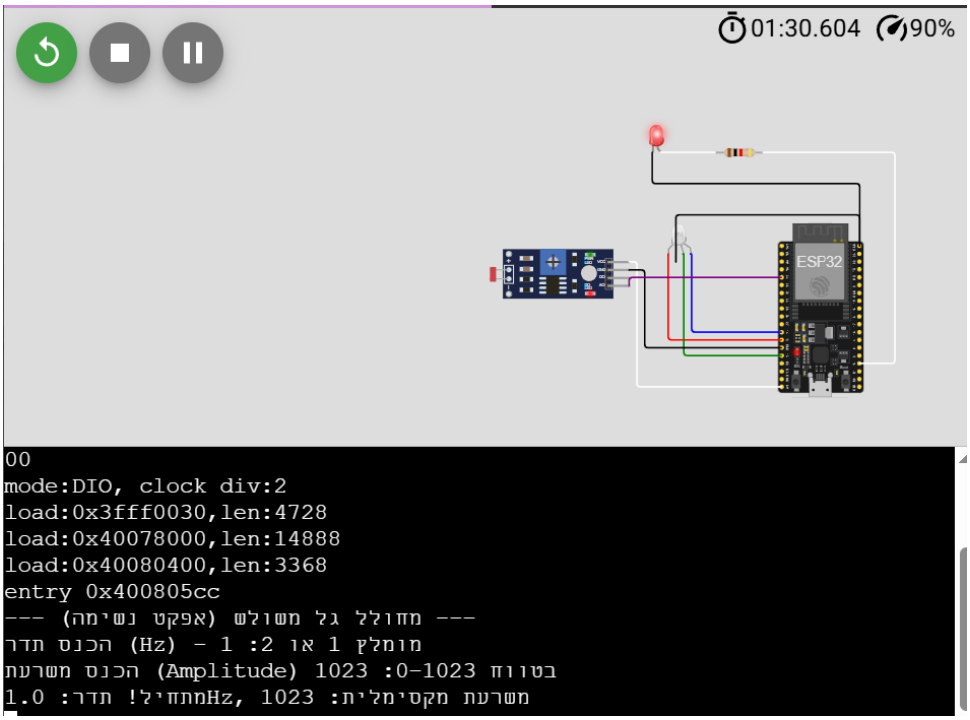

Figure 3: Serial output and LED responding to the triangular wave generator.

Source Code:

```
from machine import Pin, PWM
import time

led_pwm = PWM(Pin(15))
led_pwm.freq(5000)

print("--- Triangular Wave Generator ---")

try:
    freq = float(input("Enter frequency (Hz) [1-2 recommended]: "
        ))
    amp = int(input("Enter amplitude (0-1023): "))

    if amp > 1023: amp = 1023
    if amp <= 0: amp = 1
```

```
    if freq <= 0: freq = 1

    step_delay = 1.0 / (2 * amp * freq)
    print(f"Starting! Freq: {freq}Hz, Max Amplitude: {amp}")

    while True:
        for duty in range(0, amp + 1):
            led_pwm.duty(duty)
            time.sleep(step_delay)

        for duty in range(amp - 1, -1, -1):
            led_pwm.duty(duty)
            time.sleep(step_delay)

except ValueError:
    print("Please enter numeric values only.")
```

4 Task 4: Reverse Parking Sensor System

Description: We implemented an HC-SR04 ultrasonic sensor to measure distance. The distance is used to trigger different colors on an RGB LED, simulating a reverse parking warning system.

Expected Result: The RGB LED must light up Green if the distance is ¿20cm, Blue if between 10cm and 20cm, and Red if ¡10cm.

Circuit & Execution Proof:

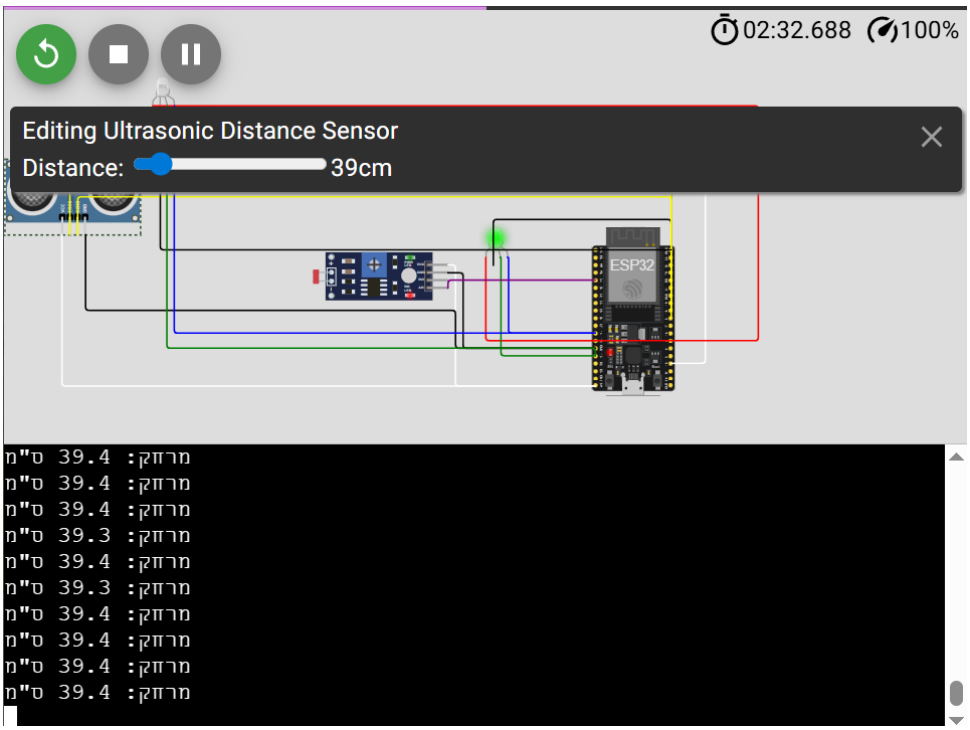

Figure 4: System testing of the ultrasonic reverse parking logic.

Source Code:

```
from machine import Pin, time_pulse_us
import time

trig = Pin(5, Pin.OUT)
echo = Pin(18, Pin.IN)

red_led = Pin(12, Pin.OUT)
green_led = Pin(13, Pin.OUT)
blue_led = Pin(14, Pin.OUT)

def get_distance():
    trig.value(0)
    time.sleep_us(2)
    trig.value(1)
    time.sleep_us(10)
    trig.value(0)

```

```python
    duration = time_pulse_us(echo, 1, 30000)

    if duration < 0:
        return -1

    distance = (duration * 0.0343) / 2
    return distance

def turn_off_all():
    red_led.value(0)
    green_led.value(0)
    blue_led.value(0)

print("--- Reverse Parking System Active ---")

while True:
    dist = get_distance()

    if dist < 0:
        print("No obstacle detected.")
        turn_off_all()
    else:
        print(f"Distance: {dist:.1f} cm")
        turn_off_all()

        if dist > 20:
            green_led.value(1)
        elif 10 <= dist <= 20:
            blue_led.value(1)
        elif dist < 10:
            red_led.value(1)

    time.sleep(0.5)
```